

Goodreads Recommendation System

Repository: <https://github.com/yoonseol-jang/goodreads-recommendation-system>

Preprocessing and Data Splitting

CSV-to-Parquet Conversion. The raw interaction file contains approximately 223 million rows. Reading it as CSV on every experiment incurs a full schema-inference pass in addition to the data scan, which is wasteful on a shared cluster. As the first step of the pipeline, we converted the file to Parquet using an explicit schema definition rather than `inferSchema=True`, avoiding the extra scan entirely. All downstream scripts read from the Parquet version, which benefits from columnar storage, predicate pushdown, and compressed I/O.

User-Based Sampling. Because full-dataset ALS and MinHash runs were too costly on the shared cluster, the final experiments use a deterministic 3% user sample. The sample was drawn by computing `xxhash64(user_id) % 100` and keeping users whose hash value falls below 3. We also used the same deterministic hash rule with a 1% threshold for early hyperparameter and weight screens, then reran selected configurations on the 3% sample for final comparisons. Because `xxhash64` is a pure function of `user_id`, every row belonging to a kept user passes the filter in a single scan with no additional distinct-and-join shuffle. The resulting sample preserves each sampled user's complete interaction history, maintaining the natural distribution of interaction counts rather than distorting it by downsampling individual records.

Train/Validation/Test Split. Recommendation evaluation requires that each user's history be divided into an observed portion, used to generate recommendations, and a held-out portion, used to score them. We first discarded users with fewer than five interactions, as they do not provide enough history to support a meaningful three-way split. For each remaining user, interactions were randomly ordered with a fixed seed (`seed=42`) and then partitioned within each user: the first 60% were assigned to training, the next 20% to validation, and the final 20% to test. Splitting within users rather than across users guarantees that every validation and test user also has a training history, eliminating cold-start cases from the evaluation cohort by construction. All three splits were written to Parquet for reuse across all subsequent experiments.

1 Market Segmentation

For market segmentation, we represented each book by the set of users who read it, using only interactions where `is_read = 1`. To reduce unstable similarities from very rare books, we filtered out books with fewer than 30 readers in the 3% user sample before computing similarity. Each book's reader set was then encoded as a binary sparse vector via `CountVectorizer (binary=True)`, and Spark `MinHashLSH` was fit on these vectors with `numHashTables=5` to produce signatures that approximate Jaccard similarity in a fixed-length, low-dimensional space.

Scalability. A brute-force search over all book pairs is $O(N^2)$ in the number of books, which is prohibitive even on the 3% sample. `MinHashLSH` avoids this by hashing each signature into buckets such that only books that collide in at least one of the 5 hash tables become candidate pairs — highly similar books collide with high probability, while dissimilar ones rarely do. We invoked `approxSimilarityJoin` with a Jaccard distance threshold of 0.8 (i.e. candidates with similarity ≥ 0.2), which is loose enough to retain all genuinely high-overlap pairs while still discarding the vast majority of the $O(N^2)$ search space at the hashing stage. The shortlist of candidate pairs is small enough that we then compute exact intersection and union sizes per pair in a single distributed pass, so MinHash supplies the recall and the exact step supplies the final ranking precision.

The final top 100 pairs were sorted by exact Jaccard similarity. The highest-ranked pair had 34 shared readers out of 35 users in the union, giving a similarity of 0.9714. Overall, the top 100 pairs had high reader overlap, with similarities ranging from about 0.97 to 0.86, suggesting that the method identified books occupying similar positions in the reading market.

2 Popularity Baseline

Popularity Score and Recommendation Generation. The popularity baseline recommends the same globally popular books to every user, personalized only by removing books the user has already seen. We defined popularity as the number of training interactions with `is_read = 1` for each book, matching the implicit consumption signal that is the most abundant feedback type in the dataset. This choice also makes the baseline directly comparable to the implicit ALS model, since both use the same behavioral signal.

To generate recommendations, we first selected the top 1,000 globally popular books. Retaining a candidate pool larger than the final top-100 provides headroom: heavy readers may have already seen many popular books, and without extra candidates they could receive fewer than 100 recommendations. For each training user, we removed all books present in their training history via a left-anti join, then retained the top 100 unseen books ranked by global read count. Recommendations were computed once over all training users and cached, so the same precomputed list was reused across both label definitions and both evaluation splits.

Evaluation. We evaluated under two label definitions to make the baseline comparable with later models. The first treats `is_read = 1` as relevance, consistent with the popularity signal. The second treats `rating  $\geq 4$` as relevance, matching the preference signal used by the explicit ALS model.

Under `is_read = 1` labels, the baseline achieved MAP 0.0320, NDCG@100 0.1081, precision@100 0.0275, and re-

Table 1: Top 100 most similar book pairs from the 3% user sample, after filtering to books with at least 30 readers. Pairs are sorted by exact Jaccard similarity; the left block lists ranks 1–50 and the right block lists ranks 51–100. r_1, r_2 are the reader counts of each book; $|\cap|, |\cup|$ are intersection and union sizes of the reader sets.

b_1	b_2	r_1	r_2	$ \cap $	$ \cup $	J	b_1	b_2	r_1	r_2	$ \cap $	$ \cup $	J
3952	3953	35	34	34	35	0.971429	21846	21848	50	54	49	55	0.890909
21738	21739	71	70	69	72	0.958333	17471	17473	52	50	48	54	0.888889
21736	21737	66	67	65	68	0.955882	21840	21843	33	35	32	36	0.888889
3958	3959	44	42	42	44	0.954545	3951	3952	33	35	32	36	0.888889
4383	4385	42	44	42	44	0.954545	21840	21841	33	35	32	36	0.888889
4383	4384	42	42	41	43	0.953488	58991	58992	119	117	111	125	0.888
58908	58910	41	43	41	43	0.953488	14229	83430	59	58	55	62	0.887097
3955	3957	39	39	38	40	0.95	8264	8268	96	102	93	105	0.885714
112530	112531	34	36	34	36	0.944444	58921	58922	72	73	68	77	0.883117
207578	207579	32	32	31	33	0.939394	13976	13977	274	289	264	299	0.882943
51199	67248	32	32	31	33	0.939394	56477	56478	32	32	30	34	0.882353
44736	44737	31	33	31	33	0.939394	58934	58935	31	33	30	34	0.882353
12433	12434	31	31	30	32	0.9375	140338	140339	31	33	30	34	0.882353
44669	44673	40	43	40	43	0.930233	113027	113028	39	40	37	42	0.880952
21737	21739	67	70	66	71	0.929577	157997	157998	31	31	29	33	0.878788
58916	58918	52	54	51	55	0.927273	5429	5430	31	31	29	33	0.878788
19320	19321	91	90	87	94	0.925532	3910	3911	152	163	147	168	0.875
89049	89051	47	47	45	49	0.918367	21847	21848	51	54	49	56	0.875
21737	21738	67	71	66	72	0.916667	5831	5833	45	45	42	48	0.875
4385	4386	44	48	44	48	0.916667	4384	4386	42	48	42	48	0.875
3953	3954	34	35	33	36	0.916667	4383	4386	42	48	42	48	0.875
21736	21739	66	70	65	71	0.915493	102964	102966	36	39	35	40	0.875
49272	49273	34	33	32	35	0.914286	4380	4381	36	39	35	40	0.875
3951	3953	33	34	32	35	0.914286	56480	56482	30	30	28	32	0.875
3949	3950	32	33	31	34	0.911765	19322	19323	86	77	76	87	0.873563
7277	7278	121	116	113	124	0.91129	21739	21740	70	78	69	79	0.873418
4384	4385	42	44	41	45	0.911111	14473	14474	56	62	55	63	0.873016
56485	56486	51	56	51	56	0.910714	49272	49274	34	39	34	39	0.871795
40051	40052	72	75	70	77	0.909091	42969	42974	56	60	54	62	0.870968
104123	104124	40	44	40	44	0.909091	14226	14227	51	50	47	54	0.87037
15127	15128	62	62	59	65	0.907692	4386	4387	48	53	47	54	0.87037
14118	14119	81	83	78	86	0.906977	15535	15537	231	242	220	253	0.869565
44669	44670	40	42	39	43	0.906977	14030	14119	89	83	80	92	0.869565
58919	58920	71	72	68	75	0.906667	32574	32575	55	59	53	61	0.868852
21846	21847	50	51	48	53	0.90566	40054	40056	92	93	86	99	0.868687
21736	21738	66	71	65	72	0.902778	3953	3956	34	37	33	38	0.868421
3955	3956	39	37	36	40	0.9	15533	15534	187	196	178	205	0.868293
3956	3957	37	39	36	40	0.9	42701	42702	262	261	243	280	0.867857
19321	19326	90	94	87	97	0.896907	5682	5684	123	131	118	136	0.867647
11089	79749	46	45	43	48	0.895833	40055	40056	90	93	85	98	0.867347
11810	125309	45	46	43	48	0.895833	39309	39312	117	122	111	128	0.867188
3958	3960	44	47	43	48	0.895833	50992	50993	42	42	39	45	0.866667
21735	21736	61	66	60	67	0.895522	13975	13976	256	274	246	284	0.866197
21732	21733	53	55	51	57	0.894737	14478	14479	88	93	84	97	0.865979
88814	88815	37	35	34	38	0.894737	13977	13978	289	295	271	313	0.865815
3954	3956	35	37	34	38	0.894737	89051	89053	47	50	45	52	0.865385
2352	2353	65	58	58	65	0.892308	89049	89053	47	50	45	52	0.865385
3952	3954	35	35	33	37	0.891892	88813	88815	34	35	32	37	0.864865
69779	69784	125	119	115	129	0.891473	6166	6168	52	58	51	59	0.864407
58909	58910	44	43	41	46	0.891304	27579	59022	259	263	242	280	0.864286

call@100 0.1607 on the validation set, and MAP 0.0322, NDCG@100 0.1087, precision@100 0.0276, recall@100 0.1601 on the test set. Under `rating` ≥ 4 labels, validation performance was MAP 0.0341, NDCG@100 0.1071, precision@100 0.0206, recall@100 0.1770, and test performance was MAP 0.0344, NDCG@100 0.1073, precision@100 0.0206, recall@100 0.1754. The close agreement between validation and test scores across both label definitions suggests the baseline is stable and not sensitive to the particular split.

Table 2: Popularity baseline evaluation results on the 3% sample.

Split	Label	MAP	NDCG@100	P@100	R@100
Val.	is_read=1	0.032	0.1081	0.0275	0.1607
Test	is_read=1	0.0322	0.1087	0.0276	0.1601
Val.	rating ≥ 4	0.0341	0.1071	0.0206	0.177
Test	rating ≥ 4	0.0344	0.1073	0.0206	0.1754

3 Explicit-Feedback ALS Recommender

Model and evaluation setup. We trained Spark ALS using only the numerical Goodreads `rating` field. Rows with `rating` = 0 were excluded because they indicate missing explicit feedback rather than negative preference; `is_read` and `is_reviewed` were not used as inputs. We used `coldStartStrategy="drop"` and evaluated top-100 recommendations with Spark MLlib’s `RankingMetrics`, after removing books already present in each user’s training history. Relevance was defined as held-out `rating` ≥ 4 , matching the explicit preference signal and the rating-label popularity baseline.

Data and tuning process. We first screened ALS hyperparameters on a deterministic 1% user sample to keep the grid affordable, then carried the selected configuration to the deterministic 3% user sample used for the final explicit-ALS comparison. The 3% sample contained 6,831,620 interactions from 26,335 users before filtering. After removing users with fewer than five interactions, 6,827,622 interac-

tions and 24,415 users remained, split into 4,086,857 train rows, 1,365,418 validation rows, and 1,375,347 test rows.

Hyperparameter screen. The 1% validation screen covered `rank` $\in \{5, 20, 50, 100\}$ and `regParam` $\in \{0.001, 0.01, 0.1, 0.5\}$, with `recPool=250` and validation NDCG@100 as the selection criterion. We evaluated 13 of 16 cells; the skipped cells were unlikely to change selection after neighboring low-rank and heavily regularized settings performed poorly. Table 3 shows the evaluated cells, sorted by NDCG@100.

Table 3: 1% validation grid for explicit ALS (`recPool=250`, `maxIter=5`). Sorted by NDCG@100.

Rank	regParam	NDCG@100	MAP	P@100	R@100
100	0.01	0.014463	0.00281	0.003379	0.034114
100	0.001	0.014431	0.002534	0.003807	0.032669
50	0.01	0.005865	0.001026	0.00174	0.013486
50	0.001	0.005863	0.000665	0.001933	0.013017
50	0.1	0.002486	0.000415	0.000567	0.006412
100	0.1	0.002212	0.000401	0.000445	0.005932
20	0.01	0.002061	0.000232	0.00077	0.004884
20	0.1	0.001412	0.000171	0.000458	0.003597
20	0.5	0.000834	0.000107	0.000197	0.002278
50	0.5	0.000809	0.0001	0.000209	0.002037
5	0.01	0.000172	0.00001	0.000086	0.000345
5	0.1	0.000168	0.00001	0.000074	0.000438
5	0.5	0.000154	0.000015	0.000042	0.000496

Increasing `rank` from 5 to 100 raises NDCG@100 by roughly two orders of magnitude, indicating that the model lacks sufficient capacity at low rank and benefits substantially from more latent dimensions on this sample. The top two configurations were `rank=100`, `regParam=0.01` (NDCG@100 = 0.014463) and `rank=100`, `regParam=0.001` (NDCG@100 = 0.014431). These scores differ by only 0.000032, so they are effectively tied; we selected `regParam=0.01` because it had the highest validation score and provided slightly stronger regularization. Performance collapses at `regParam=0.1`, suggesting that overly strong regularization erases the latent structure that high rank otherwise recovers.

3% validation and test. Since the 1% grid selected `rank=100`, `regParam=0.01`, the 3% experiment reran this setting with candidate recommendation pools `recPool` $\in \{250, 500\}$ and included lower-rank sanity checks, all with `maxIter=5`. The selected high-rank setting remained best on validation.

Table 4: 3% validation comparison for the selected ALS hyperparameters (`rank=100`, `regParam=0.01`).

recPool	MAP	NDCG@100	P@100	R@100
250	0.001363	0.006217	0.001431	0.014823
500	0.001368	0.006303	0.001529	0.01489

Using validation NDCG@100 as the criterion, `recPool=500` was slightly better than `recPool=250`; its validation RMSE was 1.683. On the 3% test split, the selected setting (`rank=100`, `regParam=0.01`, `recPool=500`, `maxIter=5`) achieved MAP = 0.00158, NDCG@100 = 0.00678, P@100 = 0.00158, and R@100 = 0.01583. RMSE was 1.679, but ranking metrics were the basis for model selection and comparison.

Table 5: 3% test-set comparison using the shared `rating` ≥ 4 relevance label.

Model	MAP	NDCG@100	P@100	R@100
Popularity	0.034398	0.107278	0.020638	0.175361
Expl. ALS	0.001581	0.006778	0.001584	0.015833

The explicit ALS model underperformed the 3% popularity baseline by a wide margin under the same `rating` ≥ 4 label. This suggests that the explicit rating graph remained too sparse at the sampled scale for ALS to beat the heavy-tailed global popularity signal. The small gain from `recPool=250` to 500 also suggests that candidate-pool truncation was not the main bottleneck. We therefore interpret this result as an empirical finding about the sampled explicit-rating signal, not as evidence that the ALS implementation failed.

4 Implicit-Feedback ALS Recommender

Implicit Feedback Signal. For this model we constructed a composite behavioral signal, deliberately excluding numerical ratings. We defined an implicit score as

$$\text{implicit_score} = \text{is_read} + 2 \times \text{is_reviewed}.$$

The coefficient of 2 on `is_reviewed` reflects the intuition that writing a review demands more active engagement than simply marking a book as read, so reviewed books are treated as stronger evidence of interest. Interactions where both fields are zero were dropped, as they carry no behavioral signal. Under this scheme, a book that was read but not reviewed receives a score of 1; a book that is both read and reviewed receives a score of 3. Any zero-score row is excluded from training entirely.

Model Training and Hyperparameter Tuning. We used Spark ALS with `implicitPrefs=True`. In this mode, Spark follows the standard implicit-feedback ALS approach, where positive interactions are treated as preferences and larger interaction values receive higher confidence. As with explicit ALS, we first screened rank and regularization on the deterministic 1% user sample, then carried the selected configuration to the 3% sample for final validation and test evaluation. The screen covered ranks $\{5, 20, 50, 100\}$ and regularization values $\{0.01, 0.1, 0.5\}$ while holding `alpha` = 15.0 and `maxIter` = 5 fixed. For evaluation, we removed books already seen in training via a left-anti join and scored the remaining top 100 against held-out `is_read` = 1 labels.

Rank 100 with `regParam` = 0.1 was the best configuration, achieving NDCG@100 of about 0.1716 on validation. The top rank-100 settings were tightly clustered: `regParam=0.1` beat `regParam=0.01` by about 0.0009 NDCG@100, and `regParam=0.5` was only slightly lower. We therefore treat the choice of `regParam=0.1` as a small validation-set advantage rather than a practically large effect; repeated user-history splits could plausibly change this ordering. The effect of regularization depended on rank: at ranks 20 and 50, higher regularization (0.5) performed best,

Table 6: 1% validation grid for implicit ALS (**alpha** = 15.0, **maxIter** = 5). Sorted by NDCG@100.

Rank	regParam	NDCG@100	MAP	P@100	R@100
100	0.1	0.171637	0.056755	0.045476	0.250553
100	0.01	0.170727	0.056483	0.044781	0.249645
100	0.5	0.170672	0.056003	0.045988	0.248146
50	0.5	0.151277	0.048491	0.039392	0.22201
50	0.1	0.150733	0.048228	0.038992	0.222169
50	0.01	0.149377	0.047633	0.038376	0.22143
20	0.5	0.144677	0.045052	0.038533	0.21183
20	0.1	0.143191	0.044465	0.037988	0.210762
20	0.01	0.139779	0.043105	0.037109	0.206289
5	0.1	0.119108	0.035218	0.03206	0.174895
5	0.5	0.1186	0.035038	0.031962	0.17405
5	0.01	0.117414	0.034639	0.031375	0.173287

while at rank 100 a moderate value (0.1) was optimal. The lowest rank (5) produced the weakest results regardless of regularization, consistent with the expectation that a large, sparse dataset benefits from richer latent representations.

3% validation and test. We then reran the selected configuration on the 3% user sample with **recPool**=500. The selected configuration was stable but not exactly identical across sample and candidate-pool settings: validation NDCG@100 was 0.171637 on the 1% sample with **recPool**=250, 0.171635 on the 3% sample with **recPool**=250, and 0.171636 on the 3% sample with **recPool**=500. Thus the apparent equality at four decimal places is a rounding effect, not literally identical outputs. For implicit ALS, **recPool**=250 was already enough to generate stable top-100 recommendations after filtering out seen books, so increasing **recPool** to 500 changed the metrics only in later decimals; explicit ALS can behave differently because it uses much sparser rating-only data, so expanding the candidate pool can slightly change the final top-100 list. On the 3% test set with **recPool**=500, results were MAP 0.0566, NDCG@100 0.1722, precision@100 0.0458, recall@100 0.2509 — nearly identical to validation, indicating the selected hyperparameters generalize without overfitting to the validation split.

Table 7: Implicit ALS final 3% evaluation (**rank** = 100, **regParam** = 0.1, **alpha** = 15.0, **recPool**=500).

Split	MAP	NDCG@100	P@100	R@100
Val.	0.056755	0.171636	0.045476	0.250547
Test	0.05664	0.172238	0.045846	0.250926

Comparison with Popularity Baseline. Compared with the popularity baseline on the test set under the same **is_read** = 1 labels (MAP 0.0322, NDCG@100 0.1087, precision@100 0.0276, recall@100 0.1601), the implicit ALS model yields substantial gains: approximately 76% relative improvement in MAP, 58% in NDCG@100, 66% in precision@100, and 57% in recall@100. These results confirm that personalization — even without any explicit preference signal — substantially outperforms a non-personalized global ranking.

Comparison with Explicit ALS. Compared with the explicit-feedback ALS model from Section 3, implicit ALS achieved much stronger ranking metrics, though under a

different relevance label. Explicit ALS on the 3% test split reached MAP 0.001581 and NDCG@100 0.006778 using **rating** ≥ 4 labels, while implicit ALS reached MAP 0.0566 and NDCG@100 0.1722 using **is_read** = 1 labels. This gap should not be interpreted as a perfectly controlled model comparison because the evaluation labels differ; rather, it shows that implicit ALS is much better aligned with the abundant read-behavior signal than explicit ALS is with the sparse rating graph.

Limitations and potential fixes. The implicit model treats reading and reviewing as positive behavioral evidence, but neither signal guarantees that the user liked the book. This can inflate performance under an **is_read** = 1 label while still recommending books that were consumed but not preferred. The review weight was also fixed at 2 rather than tuned jointly with **alpha**, so future runs should search over confidence weights, **alpha**, and **maxIter**. Finally, evaluating implicit ALS under both read-based and rating-based relevance labels would make its tradeoff against explicit ALS clearer.

5 Combined Explicit and Implicit ALS

Score transformation. The combined model trains on a single **combined_score** column built from rating, read, and review signals. For user u and book i ,

$$\text{combined_score}_{ui} = \begin{cases} r_{ui} & r_{ui} > 0 \\ 2.0 & r_{ui} = 0, v_{ui} = 1 \\ 1.0 & r_{ui} = 0, v_{ui} = 0, d_{ui} = 1 \\ \text{excluded} & \text{otherwise.} \end{cases}$$

where d_{ui} is **is_read** and v_{ui} is **is_reviewed**. Ratings are kept unchanged; read/review signals are used only when no rating is available. This avoids treating a low-rated book as a strong positive simply because it was read or reviewed. This follows the same general intuition as confidence weighting in implicit-feedback recommenders: higher-effort actions provide stronger evidence than passive consumption. ALS was run in explicit mode (**implicitPrefs** = **False**) with **ratingCol** = **combined_score** and top-100 evaluation.

Sampling and splitting. The final experiment uses the deterministic 3% user sample and the shared per-user 60/20/20 split described in the intro (seed 42, users with fewer than 5 interactions removed). Keeping validation and test users connected to training data is necessary for ALS recommendations.

Cluster-conscious design. The implementation is staged to reduce reruns on Dataproc: the 3% sample and split are saved once, each trained ALS model is persisted to HDFS, validation and test metrics are saved as CSV, and saved models are reused when only evaluation needs to be rerun. The grid was kept intentionally small rather than running a wide hyperparameter sweep, in line with the assignment guidance to sample when needed, save intermediate outputs, and avoid unnecessary recomputation.

Process justification. We first used the completed 1% runs to choose the combined-score weights, then carried the best weights to the 3% sample and tuned only **rank** and **regParam**. This keeps the search goal-oriented: pick a reasonable signal transformation cheaply, then spend the larger 3% budget on the ALS parameters most likely to affect ranking quality.

Weight selection (1% sample). The 1% weight grid was run at **rank**=50, **regParam**=0.1, **recPool**=250. Table 8 shows that the lightest weights (1.0, 2.0) won on NDCG@100, so these were carried to the 3% sweep.

Table 8: 1% weight grid for combined ALS (**rank**=50, **regParam**=0.1, **recPool**=250). Best NDCG@100 in bold.

Read	Review	MAP	NDCG@100	P@100	R@100
1	2	0.000538	0.003113	0.000702	0.008151
2	3	0.000526	0.0026	0.000548	0.006694
3	4	0.000702	0.002471	0.000451	0.005923

Rank/regularization sweep (3% sample). With the selected weights, we tuned **rank** and **regParam** on the 3% sample at **recPool**=500, **maxIter**=5 (Table 9).

Table 9: 3% rank/regularization grid for combined ALS (**read_weight**=1.0, **review_weight**=2.0, **recPool**=500, **maxIter**=5). Best NDCG@100 in bold.

Rank	regParam	MAP	NDCG@100	P@100	R@100
100	0.01	0.001654	0.007444	0.001874	0.01705
100	0.1	0.000368	0.001925	0.000384	0.005151
50	0.1	0.000311	0.001623	0.000357	0.004057

Selected configuration. **rank**=100, **regParam**=0.01, **read_weight**=1.0, **review_weight**=2.0, **recPool**=500, **maxIter**=5.

Final results. The primary relevance label is **rating** ≥ 4 ; we additionally report the test split under **is_read** = 1 for completeness (Table 10).

Table 10: Combined ALS final evaluation on the 3% sample.

Split	Label	MAP	NDCG@100	P@100	R@100
Val.	rating ≥ 4	0.001654	0.007444	0.001874	0.01705
Test	rating ≥ 4	0.001751	0.007562	0.001859	0.01704
Test	is_read = 1	0.001616	0.007505	0.002169	0.015577

Comparison with baseline and other models. Table 11 compares the combined model against the popularity baseline and explicit ALS model on the 3% test split under the shared **rating** ≥ 4 relevance label. Table 12 separately compares read-label performance against implicit ALS, because implicit ALS was evaluated under **is_read** = 1.

Table 11: 3% test-set comparison using the shared **rating** ≥ 4 relevance label.

Model	MAP	NDCG@100	P@100	R@100
Popularity	0.034398	0.107278	0.020638	0.175361
Expl. ALS	0.001581	0.006778	0.001584	0.015833
Combined ALS	0.001751	0.007562	0.001859	0.01704

Table 12: 3% test-set comparison using the shared **is_read** = 1 relevance label.

Model	MAP	NDCG@100	P@100	R@100
Popularity	0.0322	0.1087	0.0276	0.1601
Implicit ALS	0.05664	0.172238	0.045846	0.250926
Combined ALS	0.001616	0.007505	0.002169	0.015577

Among the rating-based models, the combined ALS edges out explicit-only ALS on every metric, suggesting that adding read/review signals as fallback evidence when ratings are missing recovers some additional ranking signal. Both, however, underperform the popularity baseline by roughly an order of magnitude. Under the read-label evaluation, implicit ALS is much stronger than combined ALS because its training objective and evaluation label are directly aligned, while the combined model remains an explicit-mode ALS model trained on a rating-like score.

Limitations and potential fixes. The combined model uses 3% of users; full-data training could improve overlap but would require more cluster resources. **recPool**=500 may still be too small after filtering seen books; raising it to 1000 could help recall if the cluster can sustain the cost. The grid was deliberately sparse, and exploring more rank/regularization/iteration values would likely improve results. The score transformation itself is simple; an alternative is to treat **rating** as the explicit preference signal while using **is_read** and **is_reviewed** as per-interaction confidence modifiers (Hu-Koren-Volinsky style). Finally, a hybrid model that blends ALS with a popularity prior may work better given how strong the popularity baseline is on this dataset.

Contributions and Software

Member contributions. All three members contributed jointly to data preprocessing, the train/validation/test split, the five deliverables, and the writing of this report. Yoonseol Jang focused on the explicit-feedback ALS experiments, Donggyu Kim focused on the implicit-feedback ALS experiments, and David Lee focused on the combined-feedback experiments; all members participated across each stage and reviewed the final results.

AI contributions. Generative AI was used for grammar checking, table creation, text formatting, and code refactoring. All experimental design decisions, hyperparameter choices, and result interpretations were made by the team.

Additional software. All experiments used Apache Spark (Spark SQL, Spark MLlib’s ALS, MinHashLSH, and RankingMetrics). No other libraries beyond the standard Spark distribution were required.